

MANUAL TÉCNICO DE FUNCIONAMIENTO

PLC StampPLC - adquisición de entradas, registro en SD y servidor histórico por TCP

Plataforma: M5Stack StampPLC / ESP32-S3

Firmware: Arduino sobre PlatformIO + FreeRTOS

Versión documental: 17 de septiembre de 2026

Alcance

Este documento describe el funcionamiento técnico del firmware de la PLC en su estado actual: lectura de ocho entradas digitales, detección de cambios, temporización, comunicación entre tareas, almacenamiento persistente en tarjeta SD, mantenimiento de Wi-Fi, conexión TCP y recuperación de histórico bajo demanda.

Criterio del documento

La descripción se basa en los módulos actuales customMeas, customSD, customWifi, customTCP y customTask, junto con la secuencia de inicialización integrada definida para el proyecto. Los comportamientos todavía no cubiertos por el firmware se identifican explícitamente como limitaciones o mejoras pendientes.

Nota: esta versión se ha preparado deliberadamente sin figuras ni diagramas. La información se presenta mediante texto, tablas y fragmentos de código.

Contenido

1. Objetivo funcional del PLC
2. Plataforma y entorno de ejecución
3. Organización del firmware
4. Modelo de datos
5. Adquisición de las ocho entradas digitales
6. Temporización de la adquisición
7. taskRead y generación de eventos
8. Cola de medidas en RAM
9. taskWriter y persistencia en SD
10. Exclusión mutua de la tarjeta SD
11. Subsistema Wi-Fi
12. Subsistema TCP
13. Protocolo de comandos TCP
14. Recuperación del histórico desde la SD
15. Secuencia de inicialización del sistema
16. Distribución FreeRTOS: cores y prioridades
17. Funcionamiento extremo a extremo
18. Comportamiento ante fallos y reconexiones
19. Límites técnicos actuales
20. Pruebas recomendadas
21. Referencia rápida de interfaces

1. Objetivo funcional del PLC

La PLC actúa como un registrador de eventos digitales con almacenamiento local y acceso remoto. El firmware observa ocho entradas, conserva el estado actual en un byte y genera una nueva muestra únicamente cuando detecta un cambio respecto del estado anterior.

Cada evento contiene un timestamp obtenido con millis() y el estado completo de las ocho entradas. El evento se deposita en una cola de FreeRTOS. Una tarea independiente consume la cola y lo escribe en un archivo CSV de la tarjeta SD. De forma paralela, otra tarea mantiene la conectividad de red y una tarea TCP puede atender peticiones de un servidor externo.

El diseño desacopla tres actividades que tienen tiempos de ejecución muy diferentes: adquisición rápida, escritura relativamente lenta en SD y comunicaciones de red no deterministas.

Función	Implementación actual
Adquisición	Lectura periódica del expansor AW9523B cada 10 ms.
Detección de evento	Se registra únicamente cuando cambia el byte de entradas.
Buffer intermedio	Queue FreeRTOS de 500 elementos Measurement.
Persistencia	Archivo /datos.csv en tarjeta SD.
Conectividad	ESP32-S3 en modo Wi-Fi STA.
Transporte	Cliente TCP persistente hacia un servidor configurado.
Consulta histórica	Comando GET_FROM,<timestamp>.

2. Plataforma y entorno de ejecución

La plataforma es un M5Stack StampPLC basado en ESP32-S3. El firmware se compila con PlatformIO usando el framework Arduino. FreeRTOS, integrado en el entorno del ESP32, se utiliza directamente para tareas, colas, notificaciones desde interrupción y semáforos.

Elemento	Uso
ESP32-S3	CPU de la PLC, Wi-Fi, temporizadores y ejecución FreeRTOS.
M5StamPLC	Abstracción del hardware del StampPLC y acceso al expansor de E/S.
AW9523B	Expansor de E/S del que se leen las ocho entradas digitales.
Tarjeta SD	Persistencia del histórico en /datos.csv.
WiFiClient	Socket TCP cliente hacia el servidor.
FreeRTOS	Planificación concurrente y sincronización.

El firmware no utiliza loop() como motor principal. Una vez creadas las tareas, la lógica operativa reside en FreeRTOS; loop() puede quedar prácticamente vacío o dormir periódicamente.

3. Organización del firmware

Archivo	Responsabilidad principal
customMeas.h / .cpp	Estructura Measurement, lectura física de entradas y detección de cambios.
customSD.h / .cpp	Escritura en datos.csv, consulta del tamaño y lectura por bloques.
customWifi.h / .cpp	Estructura WifiSetup, conexión Wi-Fi inicial y conexión TCP básica.
customTCP.h / .cpp	Reconexión TCP, recepción de comandos y envío del histórico.

Archivo	Responsabilidad principal
customTask.h / .cpp	Timer, cola, mutex y creación/ejecución de las tareas FreeRTOS.
main.cpp	Configuración de hardware/red y orden de inicialización.

La separación pretende mantener la lógica de cada subsistema fuera de main.cpp. El main debe limitarse a configurar la PLC y crear los recursos en un orden seguro.

4. Modelo de datos

La unidad básica de información intercambiada entre adquisición y almacenamiento es Measurement:

```
struct Measurement {
    uint32_t timestamp;
    uint8_t pinState;
};
```

timestamp contiene el valor de millis() correspondiente al cambio detectado. pinState utiliza un bit por entrada: el bit 0 representa IN1, el bit 1 representa IN2 y así sucesivamente hasta IN8 en el bit 7.

Campo	Tipo	Significado
timestamp	uint32_t	Tiempo desde arranque, en milisegundos, según millis().
pinState	uint8_t	Estado simultáneo de las ocho entradas digitales.

El estado se almacena como un byte en lugar de ocho booleanos independientes. Esto simplifica la comparación de estados, la transferencia por cola y el almacenamiento.

5. Adquisición de las ocho entradas digitales

La lectura se realiza mediante el expansor AW9523B asociado al banco B del StampPLC. Se leen dos registros de 8 bits y se combinan en un valor de 16 bits:

```
uint16_t raw =
    ((uint16_t)aw.readRegister8(0x01) << 8) |
    (uint16_t)aw.readRegister8(0x00);
```

Las ocho entradas físicas utilizadas no corresponden directamente a los bits 0-7 del registro combinado. Por ello se utiliza una tabla de mapeo:

```
constexpr uint8_t IN_PIN_LIST[8] = {4, 5, 6, 7, 12, 13, 14, 15};
```

Para cada entrada lógica *i* se comprueba el bit físico IN_PIN_LIST[*i*]. Si está activo, se pone a 1 el bit *i* de pinState; si no lo está, se limpia. El resultado final siempre expresa las entradas en el orden lógico IN1...IN8.

La función leerPines() modifica por referencia el byte suministrado. La función lecturaEntrada() llama a leerPines(), compara el valor obtenido con el estado almacenado en Measurement y sólo informa de una nueva medida cuando existe un cambio.

```
if (tempPin != meas.pinState) {
    meas.pinState = tempPin;
    meas.timestamp = millis();
    return true;
}
return false;
```

Consecuencia funcional: el sistema es orientado a eventos. No se almacena una muestra idéntica cada 10 ms; se almacena el instante en que cambia el patrón de entradas.

6. Temporización de la adquisición

La periodicidad de comprobación de entradas se genera mediante un temporizador hardware. La configuración actual divide un reloj de 80 MHz por 80 para obtener un tick de 1 microsegundo y programa una alarma periódica de 10 000 microsegundos.

```
readTimer = timerBegin(0, 80, true);
timerAttachInterrupt(readTimer, &onReadTimer, true);
timerAlarmWrite(readTimer, 10000, true);
timerAlarmEnable(readTimer);
```

Por tanto, el periodo nominal de adquisición es 10 ms, equivalente a 100 comprobaciones por segundo.

La rutina de interrupción no realiza I2C, no accede a la SD y no ejecuta lógica de aplicación. Su única responsabilidad es notificar a taskRead:

```
void IRAM_ATTR onReadTimer() {
    BaseType_t taskWoken = pdFALSE;
    vTaskNotifyGiveFromISR(taskReadHandle, &taskWoken);
    if (taskWoken == pdTRUE) {
        portYIELD_FROM_ISR();
    }
}
```

Esta decisión es importante: las operaciones I2C y de almacenamiento no son apropiadas dentro de una ISR. La ISR se mantiene corta y el trabajo real se desplaza a una tarea normal.

7. taskRead y generación de eventos

taskRead permanece bloqueada esperando una notificación del timer. No hace polling activo y, mientras no recibe la notificación, no consume CPU de forma continua.

```
while (true) {
    ulTaskNotifyTake(pdFALSE, portMAX_DELAY);

    if (lecturaEntrada(meas)) {
        xQueueSend(measurementQueue, &meas, 0);
    }
}
```

Tras cada notificación se leen las entradas. Si lecturaEntrada() devuelve false, no se genera ningún elemento. Si devuelve true, el Measurement actualizado se intenta insertar en measurementQueue.

La prioridad actual de taskRead es 3 y se fija al core 1. Dentro del firmware mostrado es la tarea de mayor prioridad, porque la adquisición es la actividad más sensible a latencias.

La llamada xQueueSend(..., 0) utiliza tiempo de espera cero. Esto significa que taskRead no se bloquea si la cola está llena. En el código actual, si la cola está completa, el evento no se guarda y el resultado de xQueueSend no se comprueba. Esta condición aparece de nuevo en la sección de límites técnicos.

8. Cola de medidas en RAM

measurementQueue desacopla la producción de eventos de su escritura física en la SD. Se crea con capacidad para 500 estructuras Measurement:

```
measurementQueue = xQueueCreate(500, sizeof(Measurement));
```

Con la estructura actual, cada elemento contiene 5 bytes lógicos de información, aunque el tamaño real en memoria puede incluir padding según la alineación del compilador. La función de la cola no es almacenar el histórico completo, sino absorber temporalmente diferencias de velocidad entre adquisición y SD.

Si la SD tarda más en una operación, taskRead puede seguir encolando eventos mientras exista espacio. taskWriter los procesará posteriormente en orden FIFO.

La cola también separa responsabilidades: taskRead no necesita conocer el formato CSV ni abrir archivos, y taskWriter no necesita conocer el AW9523B ni el temporizador.

9. taskWriter y persistencia en SD

taskWriter espera indefinidamente a que exista un Measurement en la cola. Cuando recibe uno, solicita acceso exclusivo a la SD mediante sdMutex, escribe la muestra y libera el mutex.

```
xQueueReceive(measurementQueue, &meas, portMAX_DELAY);

xSemaphoreTake(sdMutex, portMAX_DELAY);
bool flag = escribir_dato_a_SD(meas);
xSemaphoreGive(sdMutex);
```

escribir_dato_a_SD() abre /datos.csv en modo append, escribe una línea y cierra el archivo. El formato actual es:

```
<timestamp>,<pinState>
```

Ejemplo:

```
15320,4
15570,12
20115,8
```

pinState se escribe como número decimal. Por ejemplo, un valor 12 equivale a 0b00001100, es decir, bits lógicos 2 y 3 activos si se interpreta el byte directamente.

El cierre del archivo después de cada muestra favorece que los datos queden persistidos incluso si posteriormente ocurre un reinicio, a cambio de mayor sobrecarga de acceso a la SD.

10. Exclusión mutua de la tarjeta SD

La tarjeta SD puede ser utilizada por dos rutas concurrentes: taskWriter escribe nuevos eventos y taskTCP lee el histórico cuando un servidor envía GET_FROM. Para impedir que ambas operaciones accedan simultáneamente al archivo se utiliza un mutex global:

```
SemaphoreHandle_t sdMutex = nullptr;
sdMutex = xSemaphoreCreateMutex();
```

Antes de cada operación de SD, la tarea ejecuta xSemaphoreTake(sdMutex, portMAX_DELAY). Si otra tarea posee el mutex, queda bloqueada hasta que éste se libera mediante xSemaphoreGive().

El bloqueo se aplica por operación y no durante toda la transferencia histórica. taskTCP toma el mutex para obtener el tamaño del archivo y, posteriormente, lo toma y libera para cada bloque leído. De esta forma taskWriter dispone de oportunidades para insertar nuevas medidas entre bloques del histórico.

Este comportamiento es coherente con el diseño: la adquisición sigue generando eventos en RAM aunque taskWriter tenga que esperar temporalmente por la SD.

11. Subsistema Wi-Fi

La configuración compartida de red se encapsula en WifiSetup:

```
struct WifiSetup {
    IPAddress servidor;
    uint16_t puerto;
    WiFiClient client;
    const char* ssid;
    const char* password;
};
```

La estructura debe permanecer válida durante toda la vida de las tareas porque taskWifi y taskTCP reciben un puntero a ella. Por ello, en la integración final se define con duración estática/global y no como variable local que desaparezca al terminar setup().

taskWifi ejecuta el ESP32 en modo WIFI_STA. Cada dos segundos comprueba `WiFi.status()`. Si la estación no está conectada, llama a `WiFi.disconnect()`, inicia una nueva conexión mediante `WiFi.begin(ssid, password)` y espera hasta obtener `WL_CONNECTED`.

Una vez conectada, imprime la dirección IP local. Después vuelve a dormir y repite la supervisión. La tarea está fijada al core 0 con prioridad 1.

El bucle de reconexión Wi-Fi espera indefinidamente mientras no se consiga conexión. Esto no bloquea `taskRead` ni `taskWriter` porque se ejecutan en tareas independientes.

12. Subsistema TCP

El ESP32 actúa como cliente TCP. El servidor tiene una IP y un puerto configurados en `WifiSetup`. `taskTCP` mantiene el socket, procesa bytes recibidos y llama a la lógica de transferencia del histórico.

Antes de trabajar con TCP, la tarea comprueba que exista Wi-Fi. Si Wi-Fi se pierde, detiene el cliente, reinicia el estado del parser de comandos y espera 500 ms antes de volver a comprobar.

Si Wi-Fi está disponible pero `WiFiClient` no está conectado, `taskTCP` llama a `asegurar_conexion_TCP()`. Esta función detiene cualquier socket anterior y ejecuta `conectar_servidor_TCP()`. Si no se consigue la conexión, la tarea espera 1 s y vuelve a intentarlo.

Tras obtener conexión, `taskTCP` llama a `leer_datos_TCP()` y duerme 10 ms. La prioridad actual es 1 y se ejecuta en el core 0.

La reconexión TCP se gestiona en el propio firmware. Por tanto, si el servidor se reinicia o cierra la conexión, el PLC puede crear un socket nuevo sin necesidad de reiniciar la adquisición o la escritura en SD.

13. Protocolo de comandos TCP

El protocolo de aplicación es textual y orientado a líneas. Los comandos deben terminar en LF (`\n`). Si se recibe CR (`\r`), se ignora, por lo que también son aceptables finales de línea CRLF.

El buffer de comando tiene 64 bytes. Si el comando supera la capacidad útil, el firmware entra en modo `discardCommand`, descarta bytes hasta el siguiente LF y responde `ERR,COMMAND_TOO_LONG`.

Entrada del servidor	Respuesta / efecto
PING	PONG
GET_FROM,<timestamp>	Inicia transferencia del histórico desde ese timestamp.
GET_FROM,	ERR,INVALID_TIMESTAMP
Timestamp no numérico	ERR,INVALID_TIMESTAMP
Comando desconocido	ERR,UNKNOWN_COMMAND
Comando demasiado largo	ERR,COMMAND_TOO_LONG

PING existe como mecanismo simple de diagnóstico a nivel de aplicación. No sustituye a los mecanismos internos de TCP, pero permite verificar que el parser y la ruta de respuesta están operativos.

Para `GET_FROM` se utiliza `strtoul()`. La cadena se considera válida únicamente si existe al menos un número y la conversión termina exactamente al final del texto del timestamp.

14. Recuperación del histórico desde la SD

Cuando se recibe `GET_FROM,<timestamp>`, `procesarComandoTCP()` convierte el timestamp a `uint32_t` y llama a `enviar_historico()`.

La transferencia sigue los pasos siguientes:

- Se toma sdMutex y se obtiene el tamaño actual de /datos.csv mediante obtener_size_datos().
- Ese tamaño se guarda como snapshotSize y se libera el mutex.
- Se envía DATA_BEGIN,<snapshotSize>.
- Si snapshotSize es cero, se envía inmediatamente DATA_END.
- Si existen datos, se leen bloques de hasta 512 bytes desde offset 0 hasta snapshotSize.
- Cada bloque se procesa carácter a carácter para reconstruir líneas CSV.
- El primer campo de cada línea se interpreta como timestamp.
- Sólo se transmite la línea si lineTimestamp >= timestamp solicitado.
- Al terminar el snapshot se envía DATA_END.

snapshotSize congela el límite de lectura en bytes al inicio de la consulta. Si taskWriter añade nuevas líneas mientras se está enviando el histórico, esas líneas pueden existir físicamente en el archivo pero no se leen en esa transferencia porque están más allá del snapshot inicial. Una consulta posterior podrá recuperarlas.

El archivo se procesa con dos buffers: SD_READ_BUFFER_SIZE = 512 para lectura por bloques y SD_LINE_BUFFER_SIZE = 64 para reconstrucción de una línea. El parser conserva lineIndex entre bloques, de modo que una línea partida entre dos lecturas se reconstruye correctamente.

Las líneas que superen la capacidad del lineBuffer se descartan mediante discardLine. Con el formato actual timestamp,pinState, 64 bytes es muy superior a la longitud normal esperada.

Importante: DATA_BEGIN contiene el tamaño del snapshot del archivo en bytes, no el número de líneas que van a ser enviadas después del filtrado por timestamp.

15. Secuencia de inicialización del sistema

La inicialización debe respetar dependencias entre recursos. El orden técnico recomendado para la integración es el siguiente:

- Inicializar Serial para diagnóstico.
- Configurar e inicializar el hardware M5StampPLC, incluida la tarjeta SD y el expansor de E/S.
- Rellenar la estructura global WifiSetup con SSID, password, IP del servidor y puerto.
- Crear taskRead; initTaskRead() crea antes measurementQueue.
- Crear taskWriter; initTaskWriter() crea antes sdMutex.
- Crear taskWifi.
- Crear taskTCP; en este punto sdMutex ya debe existir.
- Arrancar el timer de lectura en último lugar.

El timer se inicializa al final porque, una vez habilitado, puede notificar taskRead inmediatamente. Para entonces deben existir taskReadHandle y measurementQueue. Asimismo, taskTCP no debe atender una petición histórica antes de que sdMutex haya sido creado.

El WifiSetup compartido debe ser global o tener duración estática. Las tareas reciben &configWifi como parámetro y conservan ese puntero mientras permanecen activas.

16. Distribución FreeRTOS: cores y prioridades

Tarea	Core	Prioridad	Bloqueo normal
taskRead	1	3	Notificación del timer.
taskWriter	1	2	measurementQueue / sdMutex.
taskWifi	0	1	vTaskDelay y espera de Wi-Fi.
taskTCP	0	1	vTaskDelay / socket / sdMutex durante lecturas.

La adquisición y escritura se concentran en el core 1, mientras las comunicaciones se concentran en el core 0. taskRead tiene la prioridad más alta de las tareas creadas por la aplicación; taskWriter queda por debajo y las comunicaciones usan prioridad 1.

La separación reduce la interferencia directa de las esperas de red sobre la adquisición. No obstante, ambas tareas de core 1 pueden competir por tiempo de CPU si se genera una carga muy alta, y la SD continúa siendo un recurso serializado.

17. Funcionamiento extremo a extremo

El comportamiento normal del sistema puede describirse secuencialmente de la siguiente manera:

1. El timer alcanza la alarma de 10 ms.
2. La ISR notifica a taskRead y termina rápidamente.
3. taskRead se desbloquea y lee los registros del AW9523B.
4. Los bits físicos se reordenan a un byte IN1...IN8.
5. Si el byte no ha cambiado, taskRead vuelve a esperar.
6. Si ha cambiado, se actualiza Measurement con pinState y millis().
7. El Measurement se intenta introducir en measurementQueue.
8. taskWriter recibe el elemento, obtiene sdMutex y añade una línea a /datos.csv.
9. En paralelo taskWifi mantiene la asociación Wi-Fi.
10. taskTCP mantiene o reconstruye la conexión con el servidor.
11. Si el servidor envía PING, la PLC responde PONG.
12. Si el servidor envía GET_FROM,timestamp, taskTCP lee el snapshot del CSV y transmite las líneas elegibles.
13. Mientras TCP lee la SD, taskWriter espera el mutex; taskRead puede seguir depositando eventos en la cola mientras haya espacio.
14. Al terminar cada bloque de SD, TCP libera el mutex y permite que Writer progrese.

18. Comportamiento ante fallos y reconexiones

Situación	Comportamiento actual
Wi-Fi desconectado	taskWifi intenta reconectar. taskTCP detiene el socket y espera.
Servidor TCP no disponible	taskTCP intenta conexión; si falla, espera 1 s y repite.
Servidor TCP se reinicia	Al detectar socket no conectado, el PLC crea una nueva conexión.
Error abriendo datos.csv para escritura	escribir_dato_a_SD devuelve false y taskWriter imprime un error.
Error abriendo/leyendo histórico	enviar_historico devuelve false; si el socket sigue vivo se responde ERR,HISTORY_TRANSFER o ERR,SD según el punto de fallo.
Comando mal formado	Se devuelve un ERR específico cuando existe validación para ese caso.
SD ocupada por otra tarea	La tarea solicitante espera sdMutex; no accede concurrentemente.

La adquisición no depende de que exista Wi-Fi o servidor TCP. Mientras el hardware de entradas, FreeRTOS, la cola y la SD funcionen, el registro local continúa independientemente de la red.

19. Límites técnicos actuales

Los siguientes puntos describen límites del código actual y no deben confundirse con comportamiento ya resuelto:

Límite	Implicación
Cola llena	xQueueSend se ejecuta con timeout 0 y su retorno no se comprueba; un evento puede perderse si los 500 elementos están ocupados.
Timestamp basado en millis()	uint32_t realiza rollover aproximadamente cada 49,7 días. Las comparaciones simples >= no gestionan una consulta histórica que atravesase ese rollover.
Tiempo no absoluto	El CSV guarda tiempo desde arranque, no fecha/hora UTC. Tras reinicio, millis() vuelve a comenzar.
Sin rotación de archivo	datos.csv puede crecer indefinidamente mientras exista espacio en la SD.
Sin integridad transaccional	No existe CRC, checksum ni número de secuencia por registro.
Sin autenticación TCP de aplicación	El protocolo actual acepta comandos del peer que consiga establecer la conexión prevista.
Reconexión Wi-Fi indefinida	taskWifi permanece en el bucle interno hasta conectar; no aplica backoff progresivo ni timeout global.
DATA_BEGIN semántico	El valor enviado es el tamaño del archivo snapshot en bytes, no el número de registros transmitidos.
Inicialización de recursos	Las funciones init imprimen error al fallar, pero el arranque integrado debe evitar continuar con dependencias nulas.

En una evolución industrial del firmware convendría añadir contadores de eventos perdidos, comprobación explícita de retornos, política de rotación o segmentación de logs, reloj absoluto si es requerido, y un estado global de salud de subsistemas.

20. Pruebas recomendadas

Prueba	Resultado esperado
Arranque sin servidor TCP	La adquisición y el CSV siguen funcionando; taskTCP reintenta.
Cambio de una entrada	Se añade exactamente una línea con nuevo estado y timestamp.
Entrada estable	No se generan líneas repetitivas cada 10 ms.
Cambios rápidos	Los eventos llegan en orden mientras la cola no se sature.
PING	Respuesta PONG seguida de LF.
GET_FROM,0	Se transmite todo el snapshot válido del histórico.
GET_FROM timestamp intermedio	Sólo líneas con timestamp mayor o igual.
GET_FROM futuro	DATA_BEGIN seguido de DATA_END sin líneas de datos filtradas.
Reinicio del servidor	El PLC termina detectando la pérdida y se reconecta.
Caída de Wi-Fi	TCP se detiene; tras recuperar Wi-Fi se restablece la conexión.
Consulta histórica mientras cambian entradas	No hay acceso concurrente a SD; Writer puede avanzar entre bloques TCP.
Comando >63 caracteres	ERR,COMMAND_TOO_LONG y recuperación en el siguiente LF.
Línea vacía	Se ignora.
CRLF	CR se ignora y LF termina el comando.

Para una validación de larga duración es especialmente importante forzar alta actividad de entradas y transferencias históricas simultáneas, observando el nivel máximo de ocupación de measurementQueue y cualquier error de SD.

21. Referencia rápida de interfaces

Estructuras principales

```
struct Measurement {
    uint32_t timestamp;
    uint8_t pinState;
};

struct WifiSetup {
    IPAddress servidor;
    uint16_t puerto;
    WiFiClient client;
    const char* ssid;
    const char* password;
};
```

Funciones de adquisición

Función	Propósito
leerPines(uint8_t&)	Lee AW9523B y construye el byte lógico de entradas.
lecturaEntrada(Measurement&)	Detecta cambio, actualiza estado/timestamp y devuelve true si existe evento.

Funciones de SD

Función	Propósito
escribir_dato_a_SD(Measurement&)	Añade timestamp,pinState a /datos.csv.
obtener_size_datos(size_t&)	Obtiene el tamaño actual del histórico.
leer_bloque_datos(offset, buffer, maxBytes)	Abre datos.csv, hace seek y lee un bloque.

Funciones TCP

Función	Propósito
asegurar_conexion_TCP(WifiSetup&)	Comprueba el socket y reconecta cuando es necesario.
leer_datos_TCP(...)	Parser incremental de comandos por línea.
procesarComandoTCP(...)	Implementa PING y GET_FROM.
enviar_historico(...)	Lee el snapshot de datos.csv y filtra por timestamp.

Recursos globales FreeRTOS

Recurso	Función
readTimer	Ritmo periódico de 10 ms.
measurementQueue	Buffer FIFO de Measurement.
sdMutex	Exclusión mutua de acceso a SD.
taskReadHandle	Destino de la notificación desde ISR.
taskWriterHandle	Handle de tarea de escritura.
taskWifiHandle	Handle de supervisión Wi-Fi.
taskTCPHandle	Handle de comunicaciones TCP.

Constantes de comunicación y buffers

Constante	Valor actual	Uso
TCP_COMMAND_BUFFER_SIZE	64	Buffer de comando entrante.
SD_READ_BUFFER_SIZE	512	Bloque de lectura del histórico.
SD_LINE_BUFFER_SIZE	64	Reconstrucción de una línea CSV.
measurementQueue	500 elementos	Capacidad de eventos pendientes en RAM.
Periodo del timer	10 ms	Comprobación nominal de las entradas.

Resumen operativo

En funcionamiento normal, el PLC comprueba las ocho entradas cada 10 ms, almacena únicamente los cambios, conserva esos cambios en la SD y mantiene una conexión TCP independiente para que un servidor pueda recuperar el histórico. La cola FreeRTOS desacopla adquisición y almacenamiento, mientras sdMutex garantiza que la escritura local y la lectura histórica no utilicen la SD simultáneamente.